

QKV框架

- 非自主性提示：基于环境中物体的突出性和易见性
- 自主性提示：依赖于意志控制

是否包含自主性提示可以将注意力机制和全连接或汇聚层区分开

- 查询 Q Query
 - 可以理解为自主性提示
- 值 V Value
 - 注意力机制通过注意力汇聚，将选择引导到感官输入
 - 这些感官输入称为值
- 键 K Key
 - 每个V与K配对
 - 可以理解为感官输入的非自主提示

通过设计注意力汇聚的方式，让Q（自主性提示）与K（非自主性提示）配对，引导得到最匹配的V

 注意力机制通过注意力汇聚将查询（自主性提示）和键（非自主性提示）结合在一起，实现对值（感官输入）的选择倾向

注意力汇聚：Nadaraya-Watson核回归

- 该核回归是具有注意力机制的机器学习范例
- 该核回归的注意力汇聚是 **对训练数据中输出的加权平均**
 - 从注意力的角度来看，分配给每个V的注意力**权重**取决于将V所对应的K和V作为输入的函数
 - 即attention(V)（下文为 $\alpha(q, k_i)$ ）取决于a(K, V)，即注意力评分函数，通常为K和V的相似度
 - 实际上，由于注意力权重是概率分布，所以加权和本质上是加权平均值

 计算注意力汇聚的输出为值的加权和

注意力评分函数

用数学语言描述，假设有一个查询 $\mathbf{q} \in \mathbb{R}^q$ 和 m 个“键-值”对 $(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)$ ，其中 $\mathbf{k}_i \in \mathbb{R}^k$ ， $\mathbf{v}_i \in \mathbb{R}^v$ 。注意力汇聚函数 f 就被表示成值的加权和：

$$f(\mathbf{q}, (\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)) = \sum_{i=1}^m \alpha(\mathbf{q}, \mathbf{k}_i) \mathbf{v}_i \in \mathbb{R}^v,$$

其中查询 $\mathbf{q}$ 和键 $\mathbf{k}_i$ 的注意力权重（标量）是通过注意力评分函数 a 将两个向量映射成标量，再经过softmax运算得到的：

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^m \exp(a(\mathbf{q}, \mathbf{k}_j))} \in \mathbb{R}.$$

正如下图所示，选择不同的注意力评分函数 a 会导致不同的注意力汇聚操作。下为不同操作方法

掩蔽softmax

在softmax之前，将无意义的文本序列填充为无意义特殊token，设置权重为0，仅将有意义的token参与softmax

加性注意力

早期的评分函数，适用于Q和K维度不同的情况，但是计算量大

思路

- Q和K经过各自全连接层
- 加在一起
- 过tanh激活
- 再经过全连接层得到分数

缩放点积注意力

为了计算相似度，可以直接Q和K做点积，越大则越相似

假设Q和K满足 均值=0，单位方差，则点积后均值=0，方差为d。为了确保方差始终恒定为1，防止内积爆炸，将点积除以 $\sqrt{d}$ ，则**缩放点积注意力评分函数**为

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k} / \sqrt{d}.$$

若基于n个Q和m个K-V对计算注意力，其中Q和K长度为d，V长度为v，则QKV的**缩放点积注意力**为

$$\text{softmax} \left(\frac{\mathbf{QK}^\top}{\sqrt{d}} \right) \mathbf{V} \in \mathbb{R}^{n \times v}.$$

多头注意力

与其使用单独一个注意力汇聚，可以使用独立学习得到的h组不同的线性投影（可以理解为全连接层，权重为 W_q, W_k, W_v ）来变换QKV，并行送到注意力汇聚并拼接，随后经过另一个线性变换（权重为 W_o ）。这种设计称为多头注意力，对于h个注意力汇聚输出，每个汇聚称为一个头

多头注意力：多个头连结然后线性变换

- 查询 $\mathbf{q} \in \mathbb{R}^{d_q}$ 、
 - 键 $\mathbf{k} \in \mathbb{R}^{d_k}$ 和
 - 值 $\mathbf{v} \in \mathbb{R}^{d_v}$ ，
- 每个注意力头 $\mathbf{h}_i$ ($i = 1, \dots, h$) 的计算方法为：

$$\mathbf{h}_i = f(\mathbf{W}_i^{(q)} \mathbf{q}, \mathbf{W}_i^{(k)} \mathbf{k}, \mathbf{W}_i^{(v)} \mathbf{v}) \in \mathbb{R}^{p_v},$$

其中，可学习的参数包括

- $\mathbf{W}_i^{(q)} \in \mathbb{R}^{p_q \times d_q}$ 、
- $\mathbf{W}_i^{(k)} \in \mathbb{R}^{p_k \times d_k}$ 和
- $\mathbf{W}_i^{(v)} \in \mathbb{R}^{p_v \times d_v}$ ，
- 代表注意力汇聚的函数 f 。
 - 加性注意力
 - 缩放点积注意力。

多头注意力的输出需要经过另一个线性转换，它对应着 h 个头连结后的结果，因此其可学习参数是 $\mathbf{W}_o \in \mathbb{R}^{p_o \times hp_v}$ ：

$$\mathbf{W}_o \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_h \end{bmatrix} \in \mathbb{R}^{p_o}.$$

多头注意力融合了来自于多个注意力汇聚的不同知识，这些知识的不同来源于相同的Q，K和V的不同的子空间表示。

自注意力和位置编码

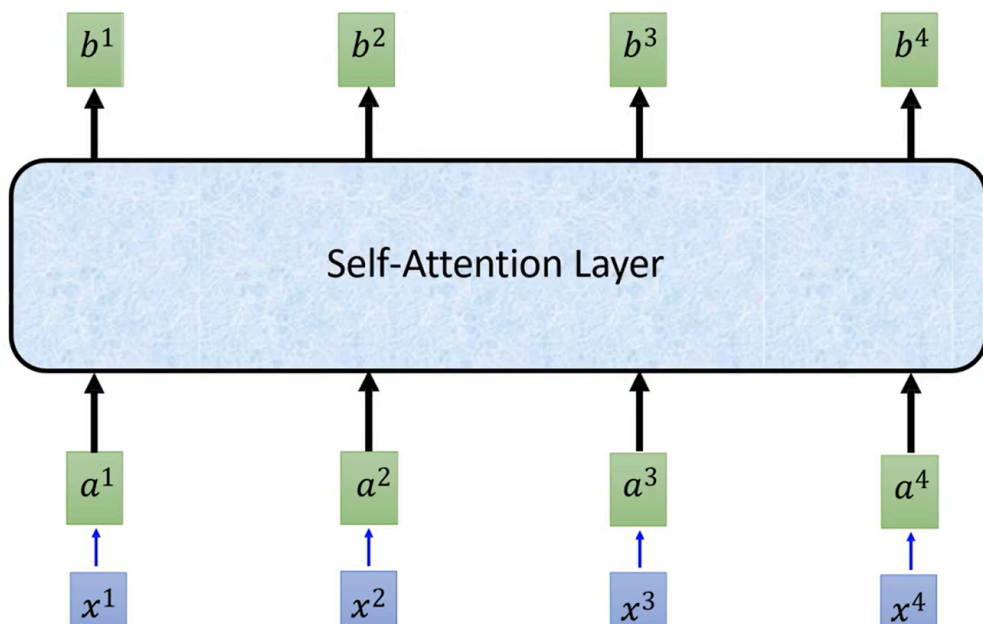
交叉注意力：QKV来自不同地方，如句子翻译

- 输入序列：[我, 爱, 猫]，提供K和V
- 翻译的词：love，提供Q

自注意力机制

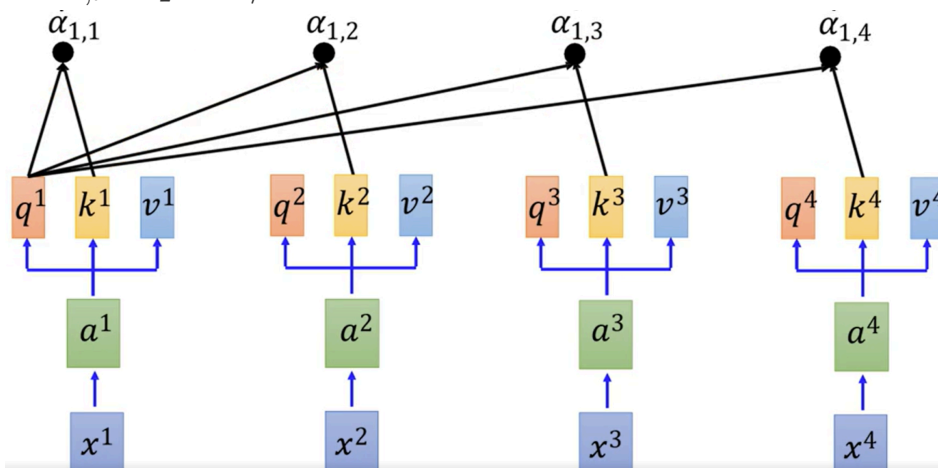
将token序列输入到注意力池中，每一组token同时充当QKV。即每个Q都会关注所有的K-V对并生成注意力输出

- 系统架构
 - 其中 $\mathbf{a}$ 为 $\mathbf{x}$ 的中间隐藏表示， $\mathbf{x}$ 做线性变换得到 $\mathbf{a}$ ， $\mathbf{a}$ 分别投影出qkv



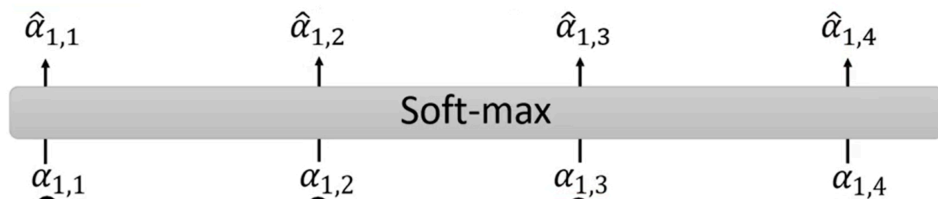
- 对于每一个b:
 - 第一步：拿每个Q去对每个K做attention，得到注意力分数

- 即 $\alpha_{1,i} = q^1 \cdot k^i / \sqrt{d}$



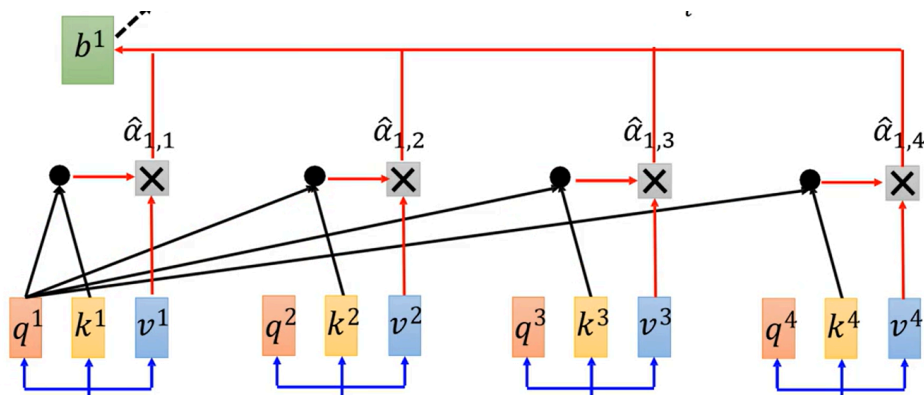
- 第二步：通过softmax得到权重

- $\hat{\alpha}_{1,i} = \text{softmax}(\alpha_{1,i})$

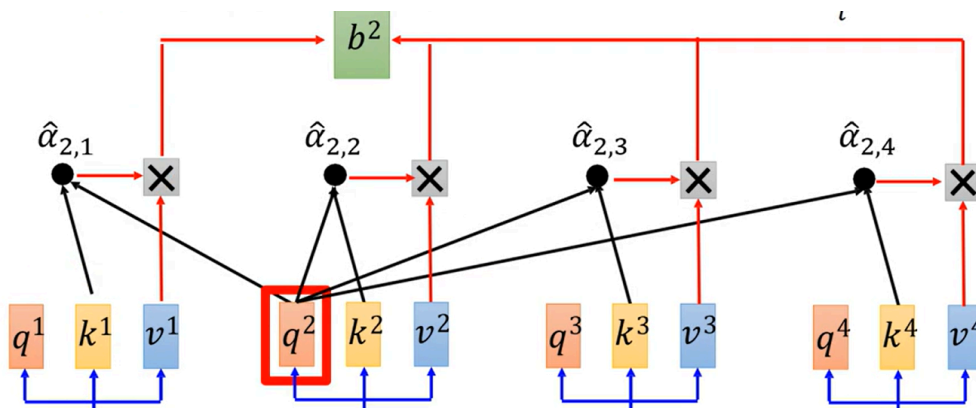


- 第三步：V和对应权重相乘并累加，得到b

- $b^1 = \sum_i \alpha_{1,i} v^i$



。同理，对于b2:



• 看似要一个一个计算，实际上已经并行计算了，如下

。所有a全部拼在一起，分别乘上QKV的矩阵，即可得到QKV

$$\text{Self-attention} \quad \begin{matrix} q^1 & q^2 & q^3 & q^4 \end{matrix} = \begin{matrix} W^q \end{matrix} \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix}$$

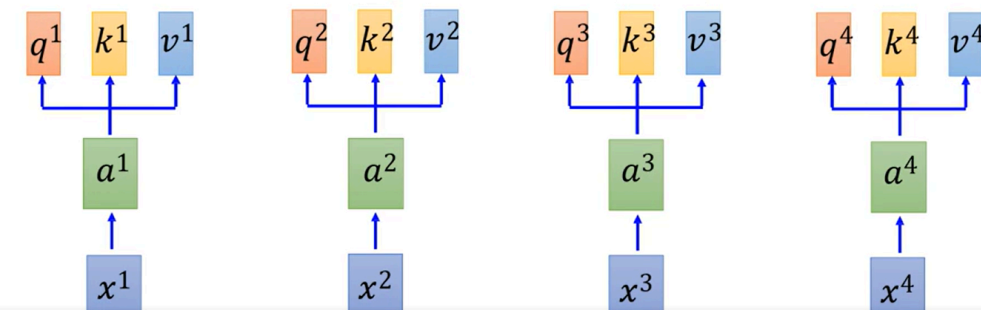
Q I

$$q^i = W^q a^i \quad \begin{matrix} k^1 & k^2 & k^3 & k^4 \end{matrix} = \begin{matrix} W^k \end{matrix} \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix}$$

K I

$$k^i = W^k a^i \quad \begin{matrix} v^1 & v^2 & v^3 & v^4 \end{matrix} = \begin{matrix} W^v \end{matrix} \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix}$$

V I



。然后将所有K拼在一起，所有Q拼在一起，相乘后再softmax得到权重

$$\hat{A} \leftarrow A = K^T Q$$

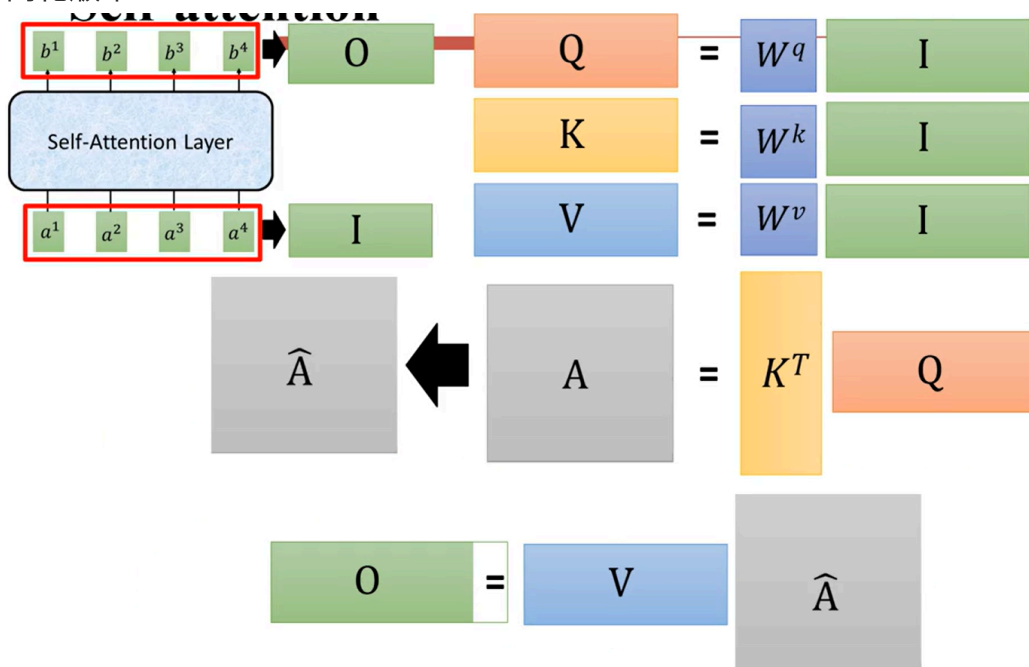
Diagram showing the calculation of the attention matrix $\hat{A}$. The input matrix A (with elements $\alpha_{i,j}$) is multiplied by the transpose of the key matrix K^T (with elements k^1, k^2, k^3, k^4) to produce the attention matrix $\hat{A}$ (with elements $\hat{\alpha}_{i,j}$). The key matrix K is derived from the input sequence Q (with elements q^1, q^2, q^3, q^4).

最后V乘以权重，得到拼在一起的b

$$O = V \hat{A}$$

Diagram showing the calculation of the output sequence O (with elements b^1, b^2, b^3, b^4). The output sequence O is calculated by multiplying the value matrix V (with elements v^1, v^2, v^3, v^4) by the attention matrix $\hat{A}$ (with elements $\hat{\alpha}_{i,j}$).

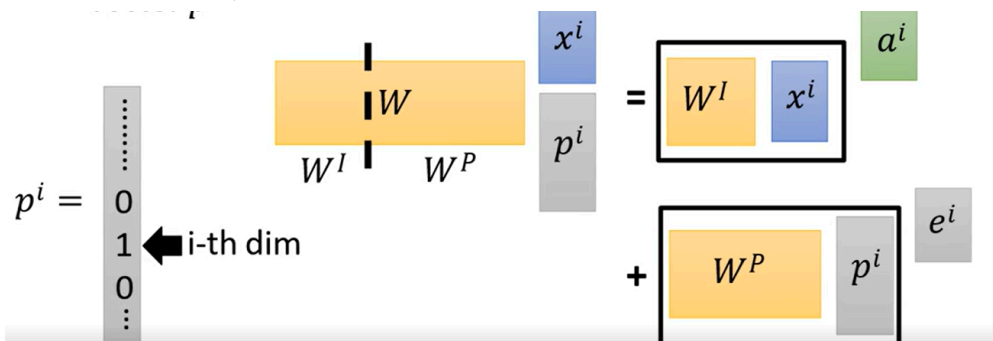
简化版本



位置编码

绝对位置编码

- 采用one-hot，如第 i 个词就 $x_i = 1$
- 若训练与测试时，见过的长度不一样则效果极差



- **相对位置编码**
 - 不关心绝对位置，只关心两个词之间的距离
 - 哪怕序列很长也不会很差

Transformer

模型

由编码器Encoder和解码器Decoder基于自注意力Self-Attention的模块叠加而成，输入（源）序列与输出（目标）序列的嵌入表示再加上位置编码，分别送到Encoder和Decoder中

transformer架构

编码器由n个相同的层叠加而成，每个层又有两个子层sublayer

- 多头自注意力
 - 基于位置的前馈神经网络
- 每个sublayer都有残差链接，且紧接着应用层规范化

解码器也由n个相同的层叠加而成，使用了残差链接和层规范化。子层有：

- 多头自注意力
- 基于位置的前馈神经网络
- **解码器-编码器注意力**

在**Decoder-Encoder Attention**中，Q来自Decoder的输出，K-V来自Encoder的输出。

而在**Decoder Self-Attention**中，QKV来自上一个Decoder的输出，一次只生成一个token，且生成第i个token时，只能看到前i-1个token，相当于掩蔽了第i个机器之后的token，防止模型在训练时偷看未来的答案，具体原理与之前的softmax掩蔽一致。

基于位置的前馈神经网络

同一个MLP，在每个位置上独立运行一次，权重在所有位置共享，可以对每个位置做非线性变换，相当于CNN的 1x1 卷积

残差连接和层规范化

批量规范化

对同一个 **batch** 的**不同**样本，在同一个特征维度上求均值和方差

即每层出口加一个自适应标准化器，先归一化（需要计算均值和方差），再拉伸偏移让数据分布稳定，加速训练

- 训练和预测不同
 - 训练时：用的是当前batch的均值和方差
 - 预测时：用的是训练阶段积累的全局均值和方差
- 小正则化效果
 - 相当于给训练添加了随机噪声

层规范化

对同一个**样本**的*所有特征*维度上求均值和方差

对比

假设数据形状为 (batch_size, features) ，例如 **3 个样本**，每个样本 **4 个特征**：

	特征1	特征2	特征3	特征4
样本1	1	2	3	4
样本2	5	6	7	8
样本3	9	10	11	12

归一化方向对比

归一化方法	计算方向	计算方式
BatchNorm	↓ 竖着算	对 每个特征 ，跨样本计算均值和方差
LayerNorm	→ 横着算	对 每个样本 ，跨特征计算均值和方差